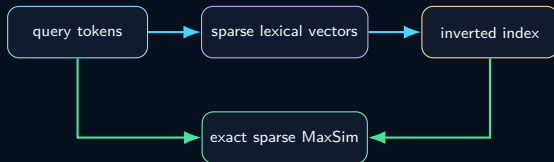


SLIM: Sparsified Late Interaction for Multi-Vector Retrieval

Reproduction → Diagnosis → Extensions

Core question. Can we keep the fine-grained accuracy of **token-level late interaction** while moving candidate generation back to a standard **inverted index**?



Harshit Raj ■ Team members
Information Retrieval Project, IIT Bombay

Primary paper: Li et al., "SLIM: Sparsified Late Interaction for Multi-Vector Retrieval with Inverted Indexes", SIGIR 2023.

1. Why SLIM exists

Sparse single-vector retrieval

BM25/SPLADE compress each document to one lexical vector.

- ▶ Excellent inverted-index compatibility.
- ▶ Cheap first-stage retrieval.
- ▶ But token-level matching structure is compressed away.

Dense late interaction

ColBERT preserves a vector per token and scores

$$S(q, d) = \sum_i \max_j q_i^\top d_j.$$

- ▶ Fine-grained token-to-token evidence.
- ▶ Much heavier multi-vector indexing/search.

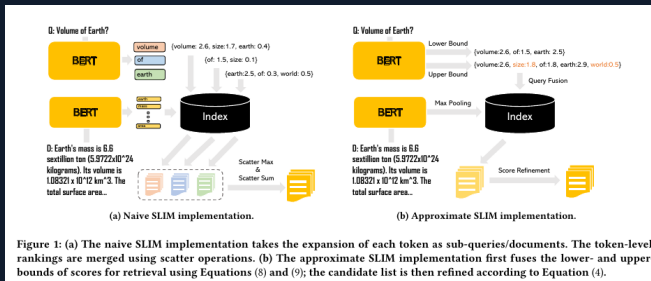
SLIM's compromise

Contextual token embeddings are projected into a shared, **non-negative sparse lexical space**.

- ▶ sparse token vectors preserve MaxSim refinement;
- ▶ document max-pooling produces one Lucene-indexable vector;
- ▶ candidate generation is approximate, refinement is exact under the sparse model.

The actual research question is not “sparse vs dense”. It is whether a **single-vector sparse first stage** can preserve enough of the candidates required by an **exact token-level second stage**.

2. SLIM in one slide — the paper's architecture



1. Sparse contextual tokens

BERT + MLM projection maps each token into a sparse vector over the vocabulary.

2. One-vector candidate retrieval

Document token vectors are max-pooled; query-side lower/upper-bound representations are fused into one sparse query vector.

3. Exact sparse late interaction

For the retrieved candidates, the original sparse token vectors are loaded and Eq. 4 is recomputed exactly.

Figure reproduced from the SLIM paper, Fig. 1.

3. The mathematics that makes SLIM indexable

Sparse lexical projection. For contextual token embedding v_{d_j} :

$$\phi_{d_j} = \log \left(1 + \text{ReLU}(W^\top v_{d_j} + b) \right) \in \mathbb{R}_{\geq 0}^{|V|}.$$

The same mapping gives query token vectors ϕ_{q_i} .

Exact sparse late interaction (paper Eq. 4)

$$S(q, d) = \sum_{i=1}^N \max_{1 \leq j \leq M} \phi_{q_i}^\top \phi_{d_j}.$$

The scorer is still MaxSim. The difference is that each token vector is now sparse and lexical.

Define the document max-pool

$$D_d = \max_j \phi_{d_j} \quad (\text{elementwise}).$$

For each query token keep only its largest lexical coordinate, giving e_{q_i} . Then the paper derives

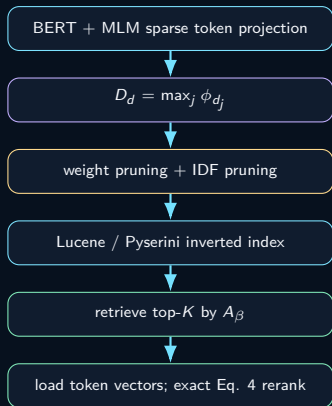
$$L(q, d) = \left(\sum_i e_{q_i} \right)^\top D_d \leq S(q, d) \leq \left(\sum_i \phi_{q_i} \right)^\top D_d = U(q, d).$$

First-stage proxy (paper Eq. 9)

$$\begin{aligned} A_\beta(q, d) &= \beta L(q, d) + (1 - \beta) U(q, d) \\ &= \left[\sum_i \left(\beta e_{q_i} + (1 - \beta) \phi_{q_i} \right) \right]^\top D_d. \end{aligned}$$

So first-stage ranking becomes **one sparse dot product per document**.

4. What actually happens at indexing and retrieval time



Paper default configuration

- ▶ token-weight threshold: 0.5;
- ▶ IDF threshold: 3;
- ▶ fusion coefficient: $\beta = 0.01$;
- ▶ Pyserini/Lucene first stage retrieves top 4000;
- ▶ exact sparse late-interaction refinement outputs top 1000.

Two different stored objects matter. Lucene only needs the pooled D_d vectors. Exact refinement additionally needs the surviving *token-level sparse vectors*; those are stored separately.

This separation is the central systems idea: candidate generation is engineered for inverted-index efficiency, while the second stage retains the richer token interaction.

5. What the original paper actually evaluates

LMV: Sparsified Late Interaction for Multi-vector Retrieval with Inverted Indexes

SLIM: 23 July 23-27, 2023, Taipei, Taiwan

Models	MARCO Dev		TREC DL19		TREC DL20		BEIR (13 tasks)		Space&Time Efficiency	
	MRR@10	R@1k	nDCG@10	R@1k	nDCG@10	R@1k	nDCG@10		Disk (GB)	Latency (ms/query)
<i>Models trained with only BM25 hard negatives from MS MARCO Passages</i>										
BM25	0.188	0.858	0.506	0.739	0.488	0.733	0.440		0.7	40
DPR	0.319	0.941	0.611	0.742	0.591	0.796	0.375		26.0	2015
SPLADE	0.340	0.965	0.683	0.813	0.671	0.823	0.453		2.6	475
COIL	0.353	0.967	0.704	0.835	0.688	0.841	0.483		78.5	3258
ColBERT	0.360	0.968	0.694	0.830	0.676	0.837	0.453		154.3	-
SLIM	0.358	0.962	0.701	0.824	0.640	0.854	0.451		18.2	580
<i>Models trained with further pre-training/hard-negative mining/distillation</i>										
coCondenser	0.382	0.984	0.674	0.820	0.684	0.839	0.420		26.0	2015
SPLADE-v2	0.368	0.979	0.729	0.865	0.718	0.890	0.499		4.1	2710
ColBERT-v2	0.397	0.985	0.744	0.882	0.750	0.894	0.500		29.0	3275
SLIM++	0.404	0.968	0.714	0.842	0.702	0.855	0.490		17.3	550

Table 1: In-domain and out-of-domain evaluation on MS MARCO Passages, TREC DL 2019/2020, and BEIR. “-” means not practical to evaluate on a single CPU. Latency is benchmarked on a single CPU and query encoding time is excluded.

Benchmarks / metrics

- ▶ MS MARCO Passage dev: MRR@10, Recall@1K;
- ▶ TREC DL19/20: nDCG@10 and recall;
- ▶ 13 BEIR tasks: average nDCG@10.

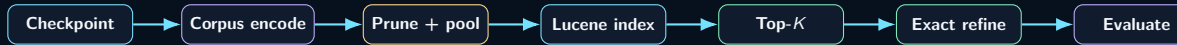
Headline numbers. SLIM reports MRR@10 0.358; SLIM++ reports 0.404. SLIM++ also reports BEIR avg. nDCG@10 0.490, 17.3 GB index size, and 550 ms/-query CPU latency in the authors' setup.



Most informative experiment. The authors vary IDF pruning and candidate depth. Exact refinement recovers much of the ranking quality lost by a cheaper first stage; the paper reports a large latency reduction for only a small MRR/recall drop.

Top: Table 1. Bottom: Fig. 2, SLIM paper.

6. Our reproduction pipeline — what must match before numbers are meaningful



Pipeline being reproduced

- ▶ released SLIM/SLIM++ checkpoint and encoder;
- ▶ sparse corpus token generation;
- ▶ document max-pooling + post-hoc pruning;
- ▶ Pyserini/Lucene candidate retrieval;
- ▶ exact sparse token refinement;
- ▶ MS MARCO effectiveness + efficiency measurements.

Controls needed for a fair comparison

- ▶ SLIM vs SLIM++ checkpoint;
- ▶ token threshold, IDF threshold, β ;
- ▶ K and whether refinement is enabled;
- ▶ Pyserini/Lucene version and index settings;
- ▶ CPU, thread count, batching, cache state;
- ▶ whether query encoding is included in latency.

Replace this line tonight with one precise sentence describing exactly what has already run successfully.

7. Reproduction result I — effectiveness

Paper vs. our run

Metric	Paper	Ours
MS MARCO MRR@10	0.358 / 0.404	[INSERT]
MS MARCO Recall@1K	0.962 / 0.968	[INSERT]
TREC DL19 nDCG@10	0.701 / 0.714	[INSERT]
TREC DL20 nDCG@10	0.640 / 0.702	[INSERT]
BEIR avg. nDCG@10	0.451 / 0.490	[INSERT]

Paper values shown as SLIM / SLIM++ where both are reported.

One-line interpretation

[e.g. "Matches the paper within X; the remaining gap is primarily first-stage candidate recall / refinement / configuration."]

Candidate recall vs. K



Also report if available: no-refinement score, refined score, and candidate recall. That immediately tells us whether a mismatch comes from candidate generation or reranking.

8. Reproduction result II — efficiency and diagnosis

Efficiency numbers

Metric	Paper	Ours
Index size	18.2 / 17.3 GB	[INSERT]
Latency	580 / 550 ms	[INSERT]
Candidate depth K	4000	[INSERT]
Refinement	on	[INSERT]

Observed bottleneck

[INSERT 1–2 precise sentences: encoding / indexing / retrieval / refinement / I/O]

If our number differs, classify the reason

- ▶ **Model/config:** checkpoint, threshold, β , K .
- ▶ **Indexing:** Pyserini/Lucene version, pruning, quantization.
- ▶ **Evaluation:** split or metric implementation.
- ▶ **Systems:** CPU, threads, batching, warm/cold cache.

Actual issue we encountered

[INSERT exact error / mismatch / engineering issue, if any]

Diagnosis rule: if effectiveness matches but latency does not, treat it as a systems/environment discrepancy—not a model failure.

9. What reproducing SLIM tells us about where to extend it

FIXED GLOBAL FUSION The paper uses one post-hoc β for all queries/tokens. It is not trained to imitate the exact late-interaction ranking.

FIXED CANDIDATE DEPTH The default system retrieves a fixed top-4000 even when an “easy” query may need far fewer candidates.

POST-HOC PRUNING Thresholds prune dimensions after training; the loss does not directly penalize long posting lists.

REFINEMENT STORAGE Exact second-stage scoring still retains document token vectors for candidates.

Design principle for our project. Do not replace SLIM wholesale. Preserve the properties that make it attractive:

- ▶ non-negative sparse lexical representations;
- ▶ standard inverted-index candidate retrieval;
- ▶ exact sparse late-interaction refinement;
- ▶ lightweight fine-tuning / offline processing only.

The four extensions below each target one concrete source of cost or mismatch, and we separate **theorem-level guarantees** from **empirical expectations**.

10. Extension 1 — certified adaptive candidate depth

Use the true SLIM upper bound for candidate ordering:

$$U(q, d) = \left(\sum_i \phi_{q_i} \right)^\top D_d, \quad S(q, d) \leq U(q, d).$$

Retrieve the first B documents by decreasing U . Let τ_k be the k -th largest *exact* score among them and θ_B the B -th upper-bound score. Then

$$\tau_k \geq \theta_B \implies \text{current top-}k \text{ is exact.}$$

Algorithm: start $B = 256$; exact-rerank; if the certificate fails, double B (512, 1024, 2048, ...) until it passes.

GUARANTEE

Every unseen document has $U(q, d) \leq \theta_B$. Since $S(q, d) \leq U(q, d)$, none can beat the current k -th exact score after $\tau_k \geq \theta_B$.

Expected effect. Easy queries stop after far fewer exact refinements; hard queries automatically spend more work. The goal is lower mean/p95 refinement cost without sacrificing top- k correctness when certified.

Caveat. The mathematical bound must survive pruning and score quantization. We therefore validate first on an unpruned/conservative score path, then add safe accounting for omitted mass if necessary.

11. Extension 2 — lossless dominance pruning of document tokens

Let $x_r, x_s \in \mathbb{R}_{\geq 0}^{|V|}$ be two sparse token vectors from the same document. If

$$x_r \leq x_s \quad \text{coordinatewise,}$$

then for every legal SLIM query token $q \geq 0$,

$$q^\top x_r \leq q^\top x_s.$$

Therefore x_r can never be a unique MaxSim winner and may be deleted.

EXACT PRESERVATION

Deleting x_r leaves $\max_j q^\top x_j$ unchanged for every non-negative q ; therefore the complete exact SLIM score is unchanged for every query.

Stage 1 is preserved too. Because $x_r[k] \leq x_s[k]$ for every coordinate k , removing x_r cannot change

$$D_d = \max_j x_j.$$

So both the Lucene representation and the exact refinement score remain identical.

Offline implementation

1. sparse pairwise dominance tests inside each document;
2. remove certified dominated tokens before writing the refinement token store;
3. optional stronger convex-dominance test only if pairwise pruning is too weak.

Only empirical question: how many tokens are actually removable, and whether the offline test is worth its preprocessing time.

12. Extension 3 — train for posting cost, not only sparsity

SLIM uses sparsity regularization and post-hoc IDF pruning. But equal nonzero counts can have very different retrieval cost: activating a dimension with a huge document frequency creates a much longer posting traversal.

Apply the cost regularizer to the *pooled vector Lucene actually indexes*. For batch $\mathcal{B}$:

$$p_k = \frac{1}{|\mathcal{B}|} \sum_{d \in \mathcal{B}} D_d^{(k)}, \quad c_k = \log(1 + \text{DF}_k),$$

$$\mathcal{L}_{DF} = \sum_k c_k p_k^2, \quad \mathcal{L} = \mathcal{L}_{rank} + \alpha \mathcal{L}_{sparse} + \gamma \mathcal{L}_{DF}.$$

Why this is aligned with the engine. The penalty is larger precisely on dimensions that create expensive long postings. It targets the representation entering Lucene rather than an abstract token-level nnz count.

Feasible training plan. Start from the released checkpoint, fine-tune briefly with the same ranking objective, sweep only a small set of γ , and plot MRR/nDCG against postings traversed, latency and index size.

EMPIRICAL Expected: fewer expensive postings and a better latency/index-size frontier. No theorem guarantees unchanged effectiveness; this is a controlled Pareto trade-off.

13. Extension 4 — importance weighting + retrieval-aware query fusion

Assign each query token a non-negative importance w_i :

$$S_w(q, d) = \sum_i w_i \max_j \phi_{q_i}^\top \phi_{d_j}, \quad w_i \geq 0.$$

The bound survives immediately:

$$\left(\sum_i w_i e_{q_i} \right)^\top D_d \leq S_w(q, d) \leq \left(\sum_i w_i \phi_{q_i} \right)^\top D_d.$$

Replace one global β by token-adaptive $\lambda_i \in [0, 1]$:

$$R(q) = \sum_i w_i \left[\lambda_i e_{q_i} + (1 - \lambda_i) \phi_{q_i} \right], \quad A(q, d) = R(q)^\top D_d.$$

The first stage remains one Lucene-compatible sparse query vector.

Minimal model. With contextual query embedding h_i :

$$w_i = \text{softplus}(a^\top h_i + b), \quad \lambda_i = \sigma(c^\top h_i + d).$$

Freeze the SLIM encoder and train only these tiny heads.

Teacher objective. Distill ranking differences from exact SLIM:

$$\mathcal{L}_m = \left[(A^+ - A^-) - (S^+ - S^-) \right]^2,$$

optionally plus a listwise KL term over a small candidate set.

EMPIRICAL Success criterion: higher exact-top- k candidate recall at smaller K ; not merely a better surrogate loss.

14. Execution plan, ablations, and what success means

1. Lock baseline

Match checkpoint/config, fill Slides 7–8, verify candidate recall vs. K .

STOP/GO No extension training until this is stable.

2. Guaranteed ideas

Implement dominance pruning + adaptive- K certification first.

LOW RISK Check exact score/ranking agreement.

3. Learned efficiency

DF-aware fine-tune; compare MRR/nDCG/Recall to postings, latency, storage.

PARETO Keep only if frontier improves.

4. Query adaptation

Train w_i, λ_i heads only after candidate-recall curves are reliable.

ABLATION global β vs adaptive λ_i .

Evaluation. MS MARCO dev + TREC DL19/20; BEIR if time permits. Report MRR/nDCG/Recall, candidate recall vs. K , mean/p95 latency, postings traversed, Lucene index size and token-store size. For guaranteed methods, additionally verify **100% score/top- k agreement** against the unmodified model under the stated assumptions.

Questions